NMOPSO in the factory patrol benchmark: preliminary results

1 NMOPSO run (benchmark) placed against suite `suite_tier1` (auction, APF, RL; 60 runs)

1 Summary

The NMOPSO run on the *benchmark* scenario completed the mission: all 6 of 6 tasks visited, with no drone excluded (no crash). It took 122.9 s and 132.0 m of flight. Its closest approach between drones was 1.28 m, and its closest approach to an obstacle was 0.57 m.

The clearest difference is speed. NMOPSO took 2.1 times as long as the slowest suite run to complete the patrol cycle. Its drones covered 0.36 m/s on average (flight distance per drone over the mission time), against 1.91 m/s for Auction, 0.75 m/s for APF, 0.78 m/s for RL (vB). Its first task was completed after 47.1 s. The data do not show whether the time went into planning, waiting or slow flight. The planner's decision time was not recorded (Section 2.3), and it is the first thing to check.

This is one run, so it supports no statistical claim. On *benchmark* the suite algorithms are very repeatable (their 95 % intervals are a few tenths of a second wide), so a gap this large is unlikely to be noise in the suite. NMOPSO's own run-to-run spread, however, is unknown, and this run could be unrepresentative. Section 4 places each NMOPSO value inside or outside the range of the other algorithms' runs. Section 5 lists what is needed for a real comparison.

- **Mission completion time:** NMOPSO 122.9 s. Range of the other algorithms' completed runs: Auction 43.7–44.3 s; APF 44.0–44.4 s; RL (vB) 58.5–59.0 s.
- **Safety:** 0 drone contacts and 0 obstacle contacts. Separation never fell below the 1 m safety radius (0 violations). Time within 0.6 m of an obstacle: 2.00 drone-s.
- **Workload:** two tasks per drone, so task imbalance is 0.00. Distance imbalance is 0.001: per-drone distances of 45.3 m, 44.7 m, 42.0 m.

2 What was run and how it was measured

2.1 NMOPSO

NMOPSO (navigation-variable multi-objective particle swarm optimisation) plans a path as n nodes. Each node is encoded in spherical coordinates relative to the previous one, so a particle is a vector of navigation variables, not of absolute waypoints:

$$\mathbf{T}_k = \mathbf{T}_{k-1}\, R_z(\phi_k)\, R_y(-\psi_k)\, T_x(r_k), \qquad \mathbf{x} = (r_1,\, \psi_1, \phi_1,\ \ldots,\ r_n,\, \psi_n, \phi_n) \tag{1}$$

A swarm of such particles is moved with the PSO velocity update:

$$\mathbf{v}_i \leftarrow w\,\mathbf{v}_i + c_1\,\mathbf{r}_1 \odot (\mathbf{p}_i - \mathbf{x}_i) + c_2\,\mathbf{r}_2 \odot (\mathbf{g} - \mathbf{x}_i), \qquad \mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{v}_i \tag{2}$$

Here $\mathrm{p_i}$ is the particle's own best position and g is a leader taken from the archive of non-dominated paths. The particles minimise four objectives together (the NMOPSO MyCost function):

$$J_1 = \left|1 - \tfrac{|\mathbf{g}-\mathbf{s}|}{L(P)}\right|, \quad J_2 = \tfrac{1}{SM}\sum_{s,\,m} \mathrm{clip}_{[0,\,1]}\!\left(1 - \tfrac{d_{sm} - \delta - R_m}{10\,\delta}\right), \quad J_3 = \tfrac{1}{n}\sum_k \tfrac{|z_k - \tilde{z}|}{\Delta z/2}, \quad J_4 = \tfrac{1}{n}\sum_k \tfrac{\angle(\mathbf{e}_k,\, \mathbf{e}_{k+1})}{\pi} \tag{3}$$

The four objectives measure:

- $\mathbf{J_1}$: length overhead against the straight line.
- $\mathbf{J_2}$: threat, i.e. proximity to obstacles modelled as cylinders of radius R_m.
- $\mathbf{J_3}$: altitude deviation.
- $\mathbf{J_4}$: turning.

The result is a Pareto front of paths. One path is then chosen from the front.

Not covered here. This describes the reference NMOPSO formulation, which is also the path model the RL planner was trained on. The parameters of the NMOPSO implementation that produced this run were not supplied with the result files, so this report does not describe them: swarm size, iterations, the archive and leader rules, how a path is picked from the front, and how the planner is called during the mission. They belong in the final report.

2.2 Measurement

The NMOPSO result files have exactly the columns of the current benchmark, including the new workload, reallocation, allocation-latency and separation-profile metrics, and the same per-run detail file. They were therefore produced by the same metric code (`metrics_summary.py`) as the suite. The battery model and the 10 Hz telemetry are also the same:

- The battery fell by 12.28 % per drone in 122.9 s. That is the platform's battery model, 0.01 % per 100 ms.
- There were 1229 valid state samples per drone, i.e. 10 Hz telemetry.

Every metric is therefore defined identically. The definitions are in `docs/METRICS.md`.

2.3 Differences in this run that affect comparability

Table 1. Differences between the NMOPSO run and the suite runs.

Observation	NMOPSO run	Suite runs	Consequence
Task visits in the mission window	6 (each task once)	9 (APF, RL) or 10 (auction): every drone visits its first task twice	NMOPSO's controller follows a different task sequence, so visit counts and idleness are not measured under identical task logic
Allocation latency	0.00 s first, 0.00 s all	about 5 s (the shared auction waits for peer states)	tasks were assigned at mission start, not by the shared auction, so latency and the AT/MCT ratio are not comparable
Decision time	not recorded	APF and RL report compute() time every 5 s	the planner's compute time is unknown. It is needed to check that planning fits the 100 ms coordination period
Date and version	2026-09-30	suite_tier1 (2026-10-23)	the run may come from an older controller or launcher version. Re-run it with the current one for the final comparison

3 Results: benchmark

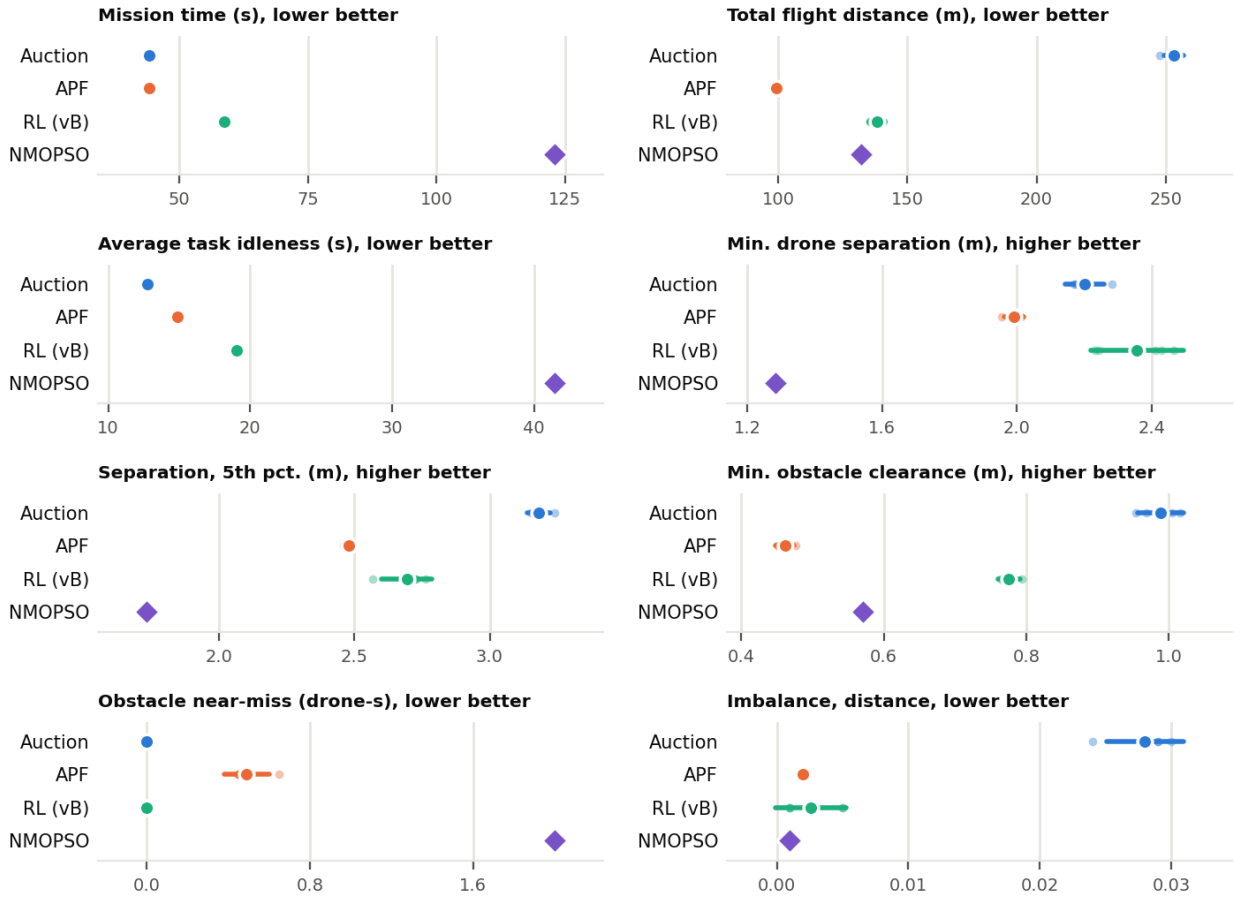


Figure 1. NMOPSO (diamond, one run) against the suite's five runs per algorithm on *benchmark*. Small dots are individual runs, and the large dot with a bar is the mean with its 95 % confidence interval. Time and distance use completed runs only.

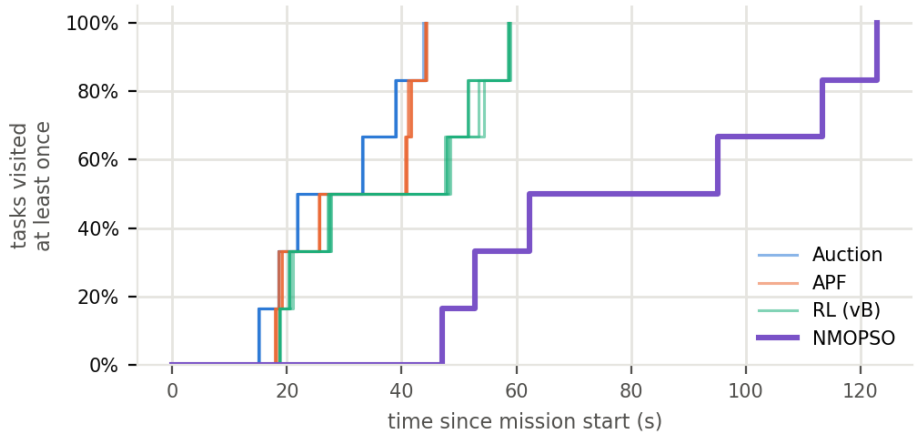


Figure 2. Coverage over time: the share of tasks visited at least once. Thin lines are individual suite runs, and the thick line is NMOPSO. The curve reaches 100 % at the mission completion time.

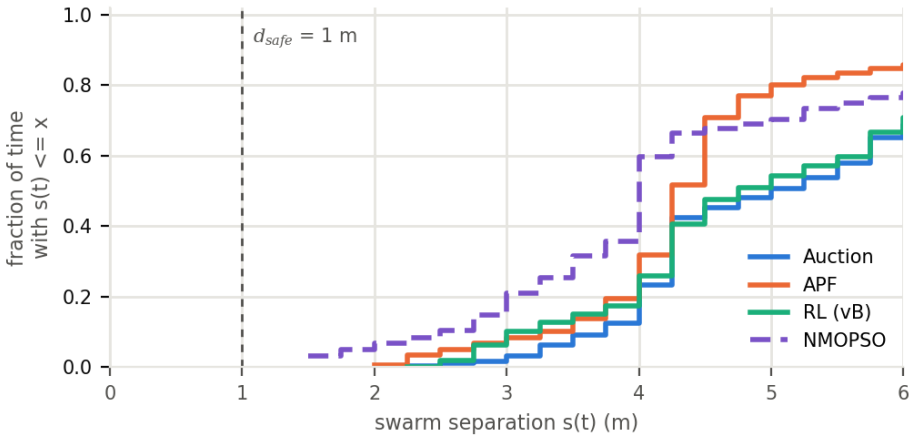


Figure 3. Distribution of the swarm separation s(t), the distance between the closest pair of drones, sampled at 20 Hz. All runs of an algorithm are pooled. A curve further right means the drones keep further apart. The dashed line is the 1 m safety radius.

4 Metric-by-metric placement: benchmark

The table shows each metric for the suite algorithms (mean ± 95 % CI over n runs) and for the single NMOPSO run. The last three columns place the NMOPSO value against each algorithm's individual runs. "better than all 5" means that no run of that algorithm did as well. With one NMOPSO run, "within range" is the expected result for any algorithm of similar performance, so it does not mean "equal".

Table 2. *benchmark*: suite means ± 95 % CI, the NMOPSO value and its placement among each algorithm's runs.

Metric	Auction	APF	RL	NMOPSO	vs Auction	vs APF	vs RL
Task completion (%)	100.0 ± 0.00	100.0 ± 0.00	100.0 ± 0.00	**100.0**	equal to all 5	equal to all 5	equal to all 5
Mission completion time (s)	44.1 ± 0.27	44.3 ± 0.17	58.7 ± 0.23	**122.9**	worse than all 5	worse than all 5	worse than all 5
Total flight distance (m)	253.0 ± 3.77	99.4 ± 0.40	138.2 ± 3.09	**132.0**	better than all 5	worse than all 5	better than all 5
Battery consumed (sum of drones) (%)	13.2 ± 0.067	13.2 ± 0.049	17.6 ± 0.064	**36.8**	worse than all 5	worse than all 5	worse than all 5
Average task idleness (s)	12.7 ± 0.018	14.9 ± 0.074	19.1 ± 0.22	**41.4**	worse than all 5	worse than all 5	worse than all 5
Worst task idleness (s)	44.1 ± 0.27	44.3 ± 0.17	58.7 ± 0.23	**122.9**	worse than all 5	worse than all 5	worse than all 5
Minimum drone separation (m)	2.20 ± 0.058	1.99 ± 0.029	2.36 ± 0.14	**1.28**	worse than all 5	worse than all 5	worse than all 5
Swarm separation, 1st percentile (m)	2.48 ± 0.097	2.02 ± 0.025	2.41 ± 0.12	**1.35**	worse than all 5	worse than all 5	worse than all 5
Swarm separation, 5th percentile (m)	3.18 ± 0.042	2.48 ± 0.016	2.69 ± 0.093	**1.74**	worse than all 5	worse than all 5	worse than all 5
Separation violations (< 1.0 m) (episodes)	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	**0.00**	equal to all 5	equal to all 5	equal to all 5
Drone near-miss time (< 1.0 m) (s)	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	**0.00**	equal to all 5	equal to all 5	equal to all 5
Drone contacts (< 0.6 m) (episodes)	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	**0.00**	equal to all 5	equal to all 5	equal to all 5
Minimum obstacle clearance (m)	0.99 ± 0.032	0.46 ± 0.013	0.78 ± 0.015	**0.57**	worse than all 5	better than all 5	worse than all 5
Obstacle near-miss time (< 0.6 m) (drone-s)	0.00 ± 0.00	0.49 ± 0.11	0.00 ± 0.00	**2.00**	worse than all 5	worse than all 5	worse than all 5
Obstacle contacts (< 0.3 m) (episodes)	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	**0.00**	equal to all 5	equal to all 5	equal to all 5
Workload imbalance, tasks (1 - Jain) (-)	0.020 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	**0.00**	better than all 5	equal to all 5	equal to all 5
Workload imbalance, distance (1 - Jain) (-)	0.028 ± 0.003	0.002 ± 0.00	0.003 ± 0.003	**0.001**	better than all 5	better than all 5	within range (0 of 5 better)
Workload Gini, distance (-)	0.091 ± 0.005	0.023 ± 0.002	0.024 ± 0.017	**0.017**	better than all 5	better than all 5	within range (3 of 5 better)
Task reallocations (count)	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	**0.00**	equal to all 5	equal to all 5	equal to all 5
Allocation latency: first assignment (s)	5.16 ± 0.025	5.16 ± 0.029	5.14 ± 0.050	**0.00**	not comparable	not comparable	not comparable
Allocation latency: all tasks assigned (s)	5.16 ± 0.025	5.16 ± 0.029	5.14 ± 0.049	**0.00**	not comparable	not comparable	not comparable
Decision time, mean (coordinator) (ms)	–	0.18 ± 0.021	9.59 ± 1.18	**–**	–	–	–

4.1 Reading the placement

- **Mission completion time** (122.9 s). Speed of one patrol cycle. Placement: Auction: worse than all 5; APF: worse than all 5; RL (vB): worse than all 5.
- **Total flight distance** (132.0 m). Path efficiency. NMOPSO optimises path length (J_1) directly. Placement: Auction: better than all 5; APF: worse than all 5; RL (vB): better than all 5.

- **Minimum obstacle clearance** (0.57 m). Closest approach to any obstacle box. Below 0.3 m the airframe touches. Placement: Auction: worse than all 5; APF: better than all 5; RL (vB): worse than all 5.
- **Minimum drone separation** (1.28 m). Closest approach between two drones. Placement: Auction: worse than all 5; APF: worse than all 5; RL (vB): worse than all 5.
- **Average task idleness** (41.4 s). How long a task waits, on average, between visits. Placement: Auction: worse than all 5; APF: worse than all 5; RL (vB): worse than all 5.

5 Conclusion and next steps

On the benchmark scenario, NMOPSO completed a full patrol cycle without a crash, a contact or a separation violation, so it works in nominal conditions. It was, however, much slower than all three suite algorithms. It also kept smaller margins than auction and RL: a smaller minimum drone separation, a smaller obstacle clearance and more time near obstacles. It still had more clearance than APF. As a patrol algorithm, this run would place it last on mission time and idleness. One run cannot rank it, though:

- NMOPSO's variance is unknown. Five runs per scenario are the minimum at which the report's Mann–Whitney tests can reach significance after Holm correction.
- The scenarios that separate the algorithms most were not run: *safety_stress* (crowded airspace, where auction and APF crash) and the two communication-loss scenarios (recovery). A factory recommendation rests mostly on these.

Needed for NMOPSO to enter the benchmark report as a fourth algorithm:

- Run 5 repeats × 4 scenarios = 20 runs with the current launcher: `./scripts/run_suite.sh suite_nmopso 5 "nmopso"`. That is about one hour.
- Report `compute_stats` from the NMOPSO planner (source `coordinator`), so its decision time is measured. Particle-swarm planning is the most expensive step of any of the four planners.
- Use the shared auction for allocation, as the other three algorithms do, or document why NMOPSO assigns at mission start. Otherwise allocation latency and comm-loss reassignment are not comparable.
- Append the NMOPSO rows to a copy of the suite manifest, then re-run `aggregate_suite.py` and `build_report.py`.